深入解析Chromium的 AI Coding 开发体系

原创 腾讯程序员 腾讯技术工程 Jun 1, 2026, 2:36 AM 广东 310 listened



作者：QQ浏览器团队出品

> 笔者作为QQ浏览器的开发，研究了一下Chromium的AI Coding开发体系，希望从中学习到一些东西。

一、整体介绍

Chromium 是全球最大的开源 C++ 项目之一，代码量超过 **3500 万行**。面对如此庞大的代码库，Chromium 团队在源码仓库中构建了一整套 **AI Agent 基础设施——不只是"用 AI 写代码"，而是建立了一套完整的提示词管理、技能系统、知识库、评估体系和大规模自动化项目。**

AI Coding 相关内容集中在 `agents/` 目录下：

```
chromium/src/agents/
├── ai_policy.md              # AI 使用政策
├── prompts/                  # 🔥 提示词系统（分层组合设计）
|   ├── common.minimal.md     # 核心指令层
|   ├── common.md             # 完整工作流层
|   ├── knowledge_base.md     # RAG 知识库
|   ├── templates/            # 平台模板层（desktop/android/ios...）
|   └── eval/                 # 评估用例（15+ 个场景）
├── skills/                   # 🔥 技能系统（18+ 个可复用技能）
├── extensions/               # MCP 扩展（工具能力增强）
├── projects/                 # AI 驱动的大型项目
└── testing/                  # Prompt 评估测试框架
```

Chromium 同时支持 Gemini CLI、Claude Code、GitHub Copilot 三种 AI 工具，核心的 Prompts 和 Skills 设计为跨工具复用。AI 使用政策（`ai_policy.md`）的核心原则是：**AI 是辅助工具，人类开发者始终是最终责任人。**如果某位开发者违反了原则，则可能被剥夺提交代码的权限。

下面逐一介绍各核心机制：**AI Policy（AI 使用政策）**、**Prompts（提示词系统）**、**Skills（技能系统）**、**Knowledge Base（知识库）**、**Eval（评估测试）** 和 **Projects（大规模项目）**。

二、AI Policy：人类始终是最终责任人

`agents/ai_policy.md` 是整个 AI Coding 体系的**底层约束**，定义了人与 AI 的责任边界。核心规则如下：

规则	要求	违规后果
自审义务	作者必须在发送 Review 前自行审查并理解所有代码，确保正确性、设计、安全性和风格达标	提交不理解的代码 → **剥夺 Committer 权限**，再犯 → **封禁账号**
原创声明	无论是否使用 AI，作者必须声明代码为自己的原创作品	—
人类回复人类	如果 AI Agent 创建的 CL 或 Bug 收到了人类的反馈，必须由人类操作者亲自回复	违反项目行为准则

此外，Policy 还给出了**推荐做法**：在 CL 描述中说明 AI 工具的使用方式（如附上 prompt）；如果是通过设计文档 + prompt 驱动 AI 生成的代码，可以将设计文档一并提交到代码库。

一句话概括：**AI 是工具，不是作者——人类开发者对每一行代码负全责。**

三、Prompts：分层组合的提示词体系

Prompts 是整个 AI Coding 体系的基石。Chromium 没有使用一个巨大的单体提示词，而是设计了一套**四层分层组合**的架构，开发者按需组合。

3.1 四层架构

```
┌──────────────────────────────────────────────┐
│  第四层：Task Prompts（任务提示词）              │
│  一次性任务指令，如 /cr:gerrit/cl-description   │
├──────────────────────────────────────────────┤
│  第三层：Templates（平台模板）                  │
│  desktop.md / android.md / ios.md / rust.md  │
├──────────────────────────────────────────────┤
│  第二层：common.md（完整工作流）                │
│  8 步标准编辑流程 + 知识库引用                  │
├──────────────────────────────────────────────┤
│  第一层：common.minimal.md（核心指令）          │
│  构建、测试、编码、JNI 等基础规范               │
└──────────────────────────────────────────────┘
```

开发者通过创建本地 `GEMINI.md` 文件，用 `@` 引用组合不同层级的 prompt：

```
# 桌面开发者的配置
@agents/prompts/common.md          # 自动包含 common.minimal.md
```

```
@agents/prompts/templates/desktop.md
```

Chromium 会使用脚本解析引用的 prompt，最终形成**一个完整的 system instruction** 注入 AI：

```
┌─────────────────────────────────────────────┐
|  System Instruction（系统指令）               |  ← GEMINI.md 递归展开后生成
|  common.minimal.md 的内容                    |
|  + common.md 追加的工作流                     |     每次对话隐式生效
|  + desktop.md 的平台规范                      |     对用户不可见
├─────────────────────────────────────────────┤
|  User Prompt（用户提示词）                    |  ← 开发者实际输入的任务
|  "帮我在 TabStrip 中实现分屏功能"             |     每次对话动态变化
└─────────────────────────────────────────────┘
```

3.2 第一层：common.minimal.md — 核心指令

这是所有开发者共享的最底层指令，定义了 AI 在 Chromium 中工作的基本规范：

规范领域	核心规则	设计意图
构建	必须先确认构建目录和目标，未确认前禁止构建	AI 猜错构建目录的代价太大，强制先确认
测试	使用 `tools/autotest.py` 运行测试，不要提前调用 `autoninja`（autotest 会自动构建）	避免重复构建
编码	Stay on task：不修无关的 TODO 和 code healt h；注释只写"为什么"不写"做了什么"	AI 有"顺手修 TODO"和"写废话注释"的倾向，需要明确约束
JNI	定义 Java↔C++ 的 JNI 方法识别规则（`@Called ByNative`、`@NativeMethods`）	即使非 Android 开发者也可能遇到 JNI 代码，所以放在最底层
预提交	完成后运行 `git cl format` + `git cl presub mit`，只修自己改动引入的问题	避免 AI 修复不相关的预存警告

3.3 第二层：common.md — 8 步标准工作流

这是 Chromium AI Coding 的核心工作流定义，**所有代码编辑任务都必须遵循**。以下是完整的 8 步流程（摘自源文件）：

```
Step 1: 深度理解代码（强制第一步，不可跳过）
  ├── 1a. 定位核心文件
  ├── 1b. 完整审计（读取每个文件的完整源码，总结控制流和所有权语义）
  ├── 1c. 陈述理解（向开发者确认理解是否正确）
  └── 1d. 反模式规避：
        • 绝不从函数名猜测行为，必须读源码实现
        • 必须检查至少一个调用点来理解用法

Step 2: 编写代码
  └── 只做需求要求的事
```

```
Step 3: 编写/更新测试
  ├── 优先向已有测试文件添加用例
  └── 没有则创建新测试文件

Step 4: 构建

Step 5: 修复编译错误
  ├── 为每个错误至少读取一个新文件
  ├── 找到并理解所有相关文件
  ├── 检查历史对话中是否出现过相同错误
  └── 绝不做投机性修复

Step 6: 运行测试

Step 7: 修复测试错误

Step 8: 迭代（循环 Step 4-7 直到全部通过）
```

其中 Step 1 的设计尤为关键——它强制 AI 在写任何代码之前，必须先**完整阅读**所有相关文件并向开发者确认理解。使用过 AI 写代码的同学应该深有体会，充分的上下文理解是保证代码质量的关键，也有助于减少幻觉问题。

3.4 第三层：Templates — 平台模板

平台模板为不同平台的开发者提供**特定的上下文和构建指令**。Chromium 提供了 desktop、android、ios 三套模板，每套都精确定义了该平台的文件命名规范、构建目标、测试运行方式，确保 AI 不会在平台特定的细节上犯错。

以桌面平台（desktop.md）为例，这是与 Windows/Mac/Linux 开发最相关的模板：

```
Before starting any tasks, you MUST read the following files:
  * //docs/chrome_browser_design_principles.md   ← 浏览器架构设计原则
  * //docs/ui/views/overview.md                  ← Views UI 框架概览

Build Targets:
  * chrome                 - 桌面 Chrome 主二进制
  * unit_tests             - 单元测试
  * browser_tests          - 集成测试
  * interactive_ui_tests   - 需要独占窗口管理器的 UI 交互测试
```

模板的关键设计在于：**强制 AI 在动手之前先阅读平台架构文档**。这确保了 AI 理解 Views 框架的 Widget/View 层级、Browser/Profile 的所有权模型等桌面平台核心概念，而不是凭猜测写代码。

3.5 第四层：Task Prompts — 自定义命令

Chromium 在 `.gemini/commands/` 中预定义了一系列可直接调用的任务提示词。每个命令本质上是一段预写好的 prompt，封装了特定环节的完整操作步骤：

命令	功能	核心逻辑
`/cr:gerrit/fix-review-comments`	自动修复 Review 意见	获取未解决评论 → 逐条判断是否有信心修复 → 修复或标记
`/cr:test/gen-gtests`	自动生成单元测试	分析 git diff → 定位测试文件 → 读取现有 fixture → 生成用例 → 验证
`/cr:gerrit/cl-description`	自动生成 CL 描述	分析 diff → 按 Chromium 规范生成标题/正文/Bug/Test 标签
`/cr:git/pre-upload-checklist`	一键预提交检查	同步依赖 → 风格检查 → 移除调试日志 → 格式化 → presubmit
`/cr:test/disable-test`	禁用失败的测试	定位测试 → 添加 DISABLED_ 前缀 → 更新 BUILD.gn

3.6 Prompt 的维护机制

Chromium 使用了一套**模板 + 注释**的维护机制：

- 源文件是 `.tmpl.md`（如 `common.tmpl.md`），包含 HTML 注释形式的设计意图说明
- 通过 `process_prompts.py` 脚本自动去除注释，生成最终的 `.md` 文件
- PRESUBMIT 检查确保 `.md` 文件与 `.tmpl.md` 保持同步

这意味着每条 prompt 规则背后都有**为什么这样写**的记录，方便团队迭代优化。

四、Skills：按需激活的专业技能

Skills 是 Chromium AI Coding 体系中的**专家模块**。与 Prompts（始终加载的通用指令）不同，Skills 是**按需激活**的——只有当用户的请求与某个 Skill 相关时，AI Agent 才会自动加载对应的 `SKILL.md` 文件。

目前 Chromium 已有 **18+ 个 Skill**，覆盖开发中的各种专业场景：

Skill	功能
`feature-flag-removal`	移除 Feature Flag（17 步完整 checklist）
`fuzzing`	编写 Fuzz 测试（含环境配置、代码模板、验证流程）
`histograms`	管理 UMA 指标（元数据、所有权、过期时间）
`cl-description`	生成 CL 描述
`git-cl-helper`	Git CL 操作辅助
`chromium-docs`	文档搜索
`network-traffic-annotations`	网络流量注解
`nullaway`	NullAway 空指针检查

Skill	功能
policy-creation	企业策略创建
webui-lit-migration	WebUI Lit 迁移
trace-analysis	性能 Trace 分析
utr	通用测试运行器
...	还有 Java 内存泄漏、JNI 类型转换、Chromebook 调试等

五、Knowledge Base：Agentic RAG 知识增强

5.1 设计理念

Chromium 的知识库不是传统的向量数据库检索，而是一套**Agentic RAG**——通过 prompt 指令让 AI 自主判断应该去读哪些文档。其核心原则写在 `knowledge_base.md` 的开头：

```
## Core Principle: Consult, then Answer

You MUST NOT answer from your general knowledge alone. The Chromium codebase is
vast and specific. Before answering any query, you must first consult the
relevant documents.
```

强制 AI 不依赖自身的通用知识，而是在回答前先去读源码中的权威文档。

5.2 三层知识增强架构

知识增强同样采用多层架构：

```
┌──────────────────────────────────────────────────────┐
|  第三层：MCP 扩展 — 外部知识源                          |
|  blink-spec (GitHub API) / build-information 等       |
├──────────────────────────────────────────────────────┤
|  第二层：chromium-docs Skill — 本地文档检索工具          |
|  2000+ md 文件索引，Python 脚本辅助 AI 定位文档          |
├──────────────────────────────────────────────────────┤
|  第一层：knowledge_base.md — 静态路由表                 |
|  任务关键词 → 文档路径的 if-then 规则引擎               |
└──────────────────────────────────────────────────────┘
```

5.3 第一层：knowledge_base.md — 静态路由表

这是最核心的机制。`knowledge_base.md` 被引用在 `common.md` 的末尾，成为 AI 上下文的一部分。它本质上是一个**任务→文档的 if-then 规则引擎**：

核心编程模式路由：

AI 检测到的任务特征	路由到的文档/动作
涉及进程间通信（browser-to-renderer）	查找对应的 `.mojom` 文件，理解 Mojo IPC 接口
涉及异步操作或线程	读取 `docs/threading_and_tasks.md`
涉及回调（`base::OnceCallback` 等）	读取 `docs/callback.md`
修改 `third_party/blink/renderer/` 下的代码	**必须** 使用 WTF 容器（`blink::Vector`、`blink::String`）和 Oilpan GC（`Member<>`、`Persistent<>`），**禁止**使用 STL 容器

功能开发路由：

AI 检测到的任务特征	路由到的文档
涉及用户偏好（pref、PrefService）	`components/prefs/README.md`
涉及 UMA 指标（histograms）	`docs/metrics/uma/README.md`
涉及 UKM 指标	`tools/metrics/ukm/README.md`
修改 BUILD.gn 文件	`docs/imported/gn/style_guide.md`

调试路由（最精细的规则）：

```
* For a "header file not found" error:
    1. Verify `deps` in BUILD.gn
    2. Verify `#include` path
    3. Regenerate build files: `gn gen <out_dir>`
    4. Confirm GN sees the dependency: `gn desc <out_dir> //failing:target deps`
    5. Check for issues: `gn check <out_dir> //failing:target`

* For a linker error ("undefined symbol"):
    → Check deps (use `gn desc`) and `is_component_build` in args.gn

* For a visibility error:
    → Add depending target to the `visibility` list
```

执行流程示例：

当开发者说"帮我在 Blink 中添加一个 CSS 属性并追踪 UMA 指标"时：

```
1. AI 分析任务关键词
2. 检测到 "Blink" → 路由规则触发：必须用 WTF 容器 + Oilpan GC
   → AI 读取 Blink C++ Style Guide
3. 检测到 "UMA 指标" → 路由规则触发
   → AI 读取 docs/metrics/uma/README.md
4. AI 基于读取的权威文档生成实现方案
```

5.4 第二层：chromium-docs Skill — 本地文档检索工具

当第一层的静态路由无法覆盖时（比如用户问了一个不在路由表中的话题），AI 可以激活 `chromium-docs` skill 进行动态搜索。

这个 Skill 实际上是一个辅助 AI 定位文档的 **Python 脚本工具**（`chromium_docs.py`）。AI 通过命令行调用它，脚本在本地 JSON 索引中做匹配，返回文档路径列表，然后 AI 自己去读取这些文档。整个流程如下：

```
AI 判断需要查文档 → 调用 python chromium_docs.py "mojo ipc"
→ 脚本在本地 JSON 索引中做字符串匹配 + 关键词匹配
→ 按权重排序，返回文档路径列表
→ AI 拿到路径后，自己去读取这些文档文件
```

索引范围：覆盖 `docs/**/*.md`、`*/README.md`、`*/docs/*.md` 等路径下的 **2000+ 个 markdown 文件**，首次构建索引约 30 秒。

本地 JSON 索引的结构：脚本在建索引阶段（`--build-index`）会扫描源码树中 2000+ 个 `.md` 文件，对每个文件提取元信息，保存为 **3 个 JSON 文件**：

索引文件	结构	用途
doc_index.json	{ "docs/threading_and_tasks.md": { title, summary, content, keywords, category, mtime }, ... }	**主索引**，存储每个文档的标题、摘要、完整内容、关键词、分类和修改时间
keyword_index.json	{ "mojo": ["docs/mojo/README.md", ...], "sandbox": [...], ... }	**关键词倒排索引**，key 是 Chromium 专有术语，value 是包含该术语的文档列表
category_index.json	{ "ui": ["docs/ui/views/overview.md", ...], "network": [...], ... }	**分类索引**，按 13 个预定义分类（如 Network Stack、User Interface 等）归类文档

其中 `doc_index.json` 中每个文档的字段通过 `_parse_document` 方法提取：标题取自 markdown 的 H1/H2 标题，摘要截取前 300 字有意义文本，关键词从内容中提取 Chromium 专有术语（最多 20 个），分类根据文件路径和内容关键词自动判定。

搜索机制：搜索时脚本遍历 `doc_index.json` 中的每条记录，用**纯字符串匹配 + 权重打分**（非向量语义检索）。标题匹配权重最高（×4.0），其次是路径匹配（×2.5）、关键词匹配（×2.0）、内容匹配（×1.0-1.5），近期修改的文档也会获得小幅加分。

最终，脚本会找到最匹配的文档，返回给 AI。

为什么需要 3 个索引而不是 1 个？

这种三分索引的设计借鉴了搜索引擎的**倒排索引 + 正排索引**架构，旨在优化不同场景下的查询效率和内存使用：

索引类型	用途	优势
`doc_index.json`	**主文档索引**，存储完整元信息用于相关性打分和结果展示	提供文档的标题、摘要、内容等完整信息用于计算匹配分数
`keyword_index.json`	**关键词倒排索引**，快速定位包含特定关键词的文档	**O(1) 时间查找**，避免遍历所有文档，速度快 1000+ 倍
`category_index.json`	**分类索引**，支持按分类浏览和过滤	**O(1) 时间获取分类文档**，支持分类维度的快速聚合

内存效率考虑：如果只有一个巨大的索引包含所有信息，每次搜索都需要将 2000+ 个文档的完整内容（可能几十MB）加载到内存。而三分方案：

- 搜索时先通过 keyword_index（很小）快速定位候选文档路径
- 然后只从 doc_index 中读取候选文档的元信息进行打分
- 内存使用量减少 90% 以上

这种设计在搜索效率和内存使用之间取得了最佳平衡，特别适合 Chromium 这种文档量大但搜索频率相对较低的场景。

索引的持久化与复用：值得注意的是，**并不需要每次搜索都重建索引**。脚本采用"一次构建，持久复用"的策略：

```
首次使用（约 30 秒）：
  python chromium_docs.py --build-index
  → 扫描 2000+ md 文件 → 生成 3 个 JSON 文件 → 持久化到磁盘

后续每次搜索（毫秒级）：
  python chromium_docs.py "mojo ipc"
  → 直接从磁盘加载已有 JSON 索引 → 内存中匹配打分 → 返回结果
```

脚本在初始化时（`__init__` → `_load_indexes()`）会尝试从磁盘读取已有的 JSON 文件，如果存在就直接使用，不会重新扫描源码树。只有当索引文件不存在时，搜索才会返回提示信息，要求开发者手动运行 `--build-index`。

不过，脚本目前**没有增量更新机制**——如果源码树中的文档发生了变化（新增、修改、删除），已有索引不会自动感知，需要手动重新运行 `--build-index` 重建。但考虑到 Chromium 的文档变化频率远低于代码变化频率，偶尔重建一次即可。

5.5 第三层：MCP 扩展 — 外部知识源

对于需要**实时外部信息**的场景，通过 MCP（Model Context Protocol）扩展获取：

- **blink-spec**：通过 GitHub API 查询 W3C/CSS 规范的 Issue 和讨论
- **build-information**：查询当前构建配置和状态
- **depot-tools**：获取 depot_tools 相关的命令帮助

5.6 与传统 RAG 的对比

维度	传统 RAG	Chromium 的 Agentic RAG
检索方式	用户 query → 向量检索 → 返回 chunks	AI 自主判断 → 按规则读取文件 → 按需搜索
知识来源	预构建的向量数据库	源码树中的原始文档（实时读取）
路由机制	纯语义相似度	静态规则表 + 动态搜索 + MCP 外部查询
更新方式	需要重新 embedding	文档随代码同步更新，索引按需重建
核心理念	被动检索	**AI 主动查阅** （"Consult, then Answer"）

六、Eval：AI Agent 的评估测试套件

Chromium 不仅构建了 AI Coding 的基础设施，还为其建立了一套**回归测试体系**——`agents/prompts/eval/` 目录。当系统提示词（如 `common.md`）发生修改时，团队可以用这些评估用例验证 AI Agent 的表现是否退化。

6.1 评估用例概览

`eval` 目录下有 **15 个子目录**，每个代表一个独立的评估任务，覆盖了日常开发中的典型场景：

评估用例	任务类型	说明
`adapt_builder`	构建配置	参照已有 LUCI builder 创建新 builder
`add_browser_test_coverage`	测试生成	为代码添加浏览器测试覆盖
`add_gtest_coverage`	测试生成	为指定方法添加单元测试
`class_refactor`	重构	将嵌套结构体重构为独立类
`fix_broken_test`	修复测试	编译、定位并修复失败的测试
`fuzzing`	Fuzz 测试	编写 FUZZ_TEST
`cl-description`	CL 描述	生成符合规范的 CL 描述
`feature_flags_add`	Feature Flag	添加新的 Feature Flag
`find_function` `/search_class`	代码搜索	查找函数定义、类定义
`build_file` `/build_target`	构建	构建指定文件或目标

6.2 用例结构

每个评估用例由两个核心文件组成：

`prompt.md` — 模拟用户输入的任务指令。例如 `fix_broken_test`：

```
I have a broken test called "DummyTest" in
third_party/blink/renderer/core/css/css_math_expression_node_test.cc.
Can you compile and run the test to figure out why it is failing,
then attempt to fix it?
```

`eval.md` 或 **`eval.promptfoo.yaml`** — 评估标准。`eval.md` 记录预期结果（修改了哪些文件、达成了什么效果），`eval.promptfoo.yaml` 则是基于 **promptfoo** 框架的**自动化断言配置**，可以精细检查：

```yaml
tests:
  - assert:
    # 检查是否修改了正确的文件
    - type: python
      value: check_files_changed
      config:
        files:
        - services/network/public/cpp/schemeful_site_mojom_traits_unittest.cc
    # 检查文件内容：必须包含 FUZZ_TEST，不能包含旧式 API
    - type: python
      value: check_file_content
      config:
        files:
          - path: '...unittest.cc'
            present: ['FUZZ_TEST', 'SchemefulSite']
            absent: ['LLVMFuzzerTestOneInput']
    # 检查是否调用了正确的构建命令
    - type: python
      value: check_tool_used_with_args_match
      config:
        tool_names: [run_shell_command]
        args_regexes: ['autoninja', 'out/fuzz']
```

6.3 设计理念

`eval` 目录的本质是一套 **AI Agent 的"单元测试"**：

- **回归测试**：修改提示词后跑一遍所有 eval 用例，确保 AI 行为没有退化
- **可复现**：每个用例绑定特定的 `Git-Revision`，可在同一代码版本上重复验证
- **自动化**：通过 promptfoo 框架可在 CI 上自动运行，检查文件变更、内容断言、工具调用等
- **参考模板**：这些用例同时也是开发者的任务参考——遇到类似任务时，可以直接参考对应的 `prompt.md`

6.4 测试框架：agents/testing/

`eval/` 目录存放的是测试用例，而 `agents/testing/` 则是**执行这些用例的测试框架**——两者的关系类似于 test_*.py 文件与 pytest 框架。

核心组件：

组件	职责
`eval_prompts.py`	主入口脚本，一键发现、过滤、调度、执行所有评估用例
`gemini_provider.py`	promptfoo 自定义 Provider，驱动 Gemini CLI 执行测试（写入 GEMINI.md → 设置 system prompt → stdin 传入 user prompt → 流式捕获输出）
`workers.py`	并行执行引擎，每个 Worker 创建独立的 WorkDir（btrfs 快照秒级创建，确保测试隔离）
`asserts/`	自定义断言脚本（check_changes.py 检查文件变更、check_tool_calls.py 检查工具调用等）

关键设计：

- **隔离执行**：每个测试在独立的 WorkDir 中运行（btrfs 快照或 gclient-new-workdir），测试之间互不干扰
- **Pass@K 机制**：一个用例运行 N 次，只要 K 次通过就算成功，适应 LLM 输出的非确定性
- **CI 级基础设施**：支持 Swarming 分片并行、Docker 沙箱隔离、ResultDB 上报、Skia Perf 性能看板，可在 CI 上自动运行
- **Telemetry 采集**：从 Gemini CLI 的 OpenTelemetry 输出中提取 token 用量和工具调用记录，用于性能追踪和断言验证

七、Projects：AI 驱动的大规模代码改造

`agents/projects/` 目录存放的不是实战案例，而是**真正在生产中运行的 AI 驱动的大规模工程项目**。与 Skills（解决单个任务）不同，Projects 面向长期的工程治理目标，包含完整的 SKILL.md + 参考文档 + Python 脚本 + 自动化流水线。

目前有 3 个项目：

项目	目标	核心机制
bedrock/modularize-chrome-browser	将 `chrome/browser/` 的巨型单体构建目标拆分为独立模块	6 阶段流程：盘点文件 → 选择构建模式 → 生成 BUILD.gn → 更新父级 BUILD → 修复 include 错误 → 验证提交。SKILL.md 长达 344 行
code-health/	代码健康自动化治理框架	含 Hub 调度中心 + 子任务（histogram-cleanup 清理过期指标、lint-sync 添加枚举同步守卫）。AI 自动发现候选项 → 置信度评估 → 创建分支 → 修改代码 → 验证 → 提交到 Gerrit
modernization/	代码现代化自动修复	Python 框架 `AutoFixer`，将错误信息喂给 Gemini 自动修复，验证后重试，最多 3 轮循环

Projects 与 Skills 的定位差异：

维度	Skills	Projects
粒度	单个任务（如"移除一个 Feature Flag"）	大规模工程项目（如"模块化整个 chrome/browser"）
自动化程度	AI 辅助，人类确认	高度自动化，含发现脚本、验证流水线、自动提交
复杂度	一个 SKILL.md	完整项目结构（SKILL.md + 参考文档 + Python 脚本 + 测试）

八、具体案例：以"实现页面分屏"为例

下面以一个具体的产品需求——"实现页面的分屏"——来完整推演 Prompts、Skills、Knowledge 如何协同工作。

> 仅推演，不代表Chromium开发中的真实工作流。

8.1 需求拆解（人类主导）

产品经理提出："在桌面版 Chrome 中，支持同一窗口内左右并排显示两个 Tab"。

人类工程师将其拆解为多个 CL（CL 即 ChangeList，等同于 GitHub 的 Pull Request）

> 以下 CL 拆解是从实际代码提交记录逆向推演的。实测中，直接将产品需求输入 AI，AI 并不能自主完成这样的拆解。下面的内容记录的是AI如何处理这些CL的。

```
CL 1: 添加 Feature Flag (kSplitView)
CL 2: 修改窗口布局模型 (BrowserView 支持双 ContentsWebView)
CL 3: 实现分屏控制器 (SplitViewController)
CL 4: 添加 UI 入口 (Tab 右键菜单 + 工具栏按钮)
CL 5: 添加 UMA 指标追踪
CL 6: 编写 Browser Tests
```

8.2 CL 1：添加 Feature Flag — Prompts + Skills 协同

开发者告诉 AI：

```
帮我添加一个 feature flag 叫 kSplitView,
放在 //chrome/browser 组件下, 默认 DISABLED_BY_DEFAULT,
需要暴露到 about:flags。
```

Prompts 层面：`common.md` 的 8 步工作流启动，AI 先进入 Step 1（深度理解）。

Knowledge 层面：`knowledge_base.md` 中没有直接的 Feature Flag 路由规则，但 `eval/feature_flags_add/` 目录下有预定义的评估用例，AI 可以参考其中的 prompt 模式。

Skills 层面：虽然没有"添加 Feature Flag"的 Skill，但有 `feature-flag-removal` Skill 可以反向参考。AI 知道最终需要修改的文件包括：

```
chrome/browser/about_flags.cc
chrome/browser/flag_descriptions.h / .cc
chrome/browser/flag-metadata.json
tools/metrics/histograms/enums.xml
```

8.3 CL 3：实现分屏控制器 — Knowledge 深度参与

开发者给 AI 技术描述后，三层知识增强机制同时工作：

第一层（knowledge_base.md 静态路由）：

检测到的特征	触发的路由	AI 的动作
代码在 chrome/browser/ui/views/	desktop.md 模板	读取 docs/ui/views/overview.md 和 docs/chrome_browser_design_principles.md
需要修改 BUILD.gn	knowledge_base 规则	读取 docs/imported/gn/style_guide.md

第二层（chromium-docs 本地检索工具）：

如果 AI 需要了解 BrowserView 的布局机制但静态路由没有覆盖，它会调用脚本搜索：

```
python chromium_docs.py "BrowserView layout ContentsWebView"
# 脚本在本地索引中匹配，返回：chrome/browser/ui/views/frame/README.md 等
# AI 再自行读取这些文档
```

第三层（Prompts 工作流）：

AI 按 8 步工作流执行：

1. 读取 `browser_view.h`、`contents_web_view.h`、`tab_strip_model.h` 的完整源码
2. 向开发者陈述理解并确认
3. 编写 `split_view_controller.h/.cc`
4. 修改 `browser_view.h/.cc` 和 `BUILD.gn`
5. 构建 → 修复编译错误 → 测试 → 修复测试错误 → 迭代

8.4 CL 5：添加 UMA 指标 — Skills 主导

开发者说："给分屏功能添加 UMA 指标追踪"。

Skills 层面：`histograms` Skill 自动激活，指导 AI：

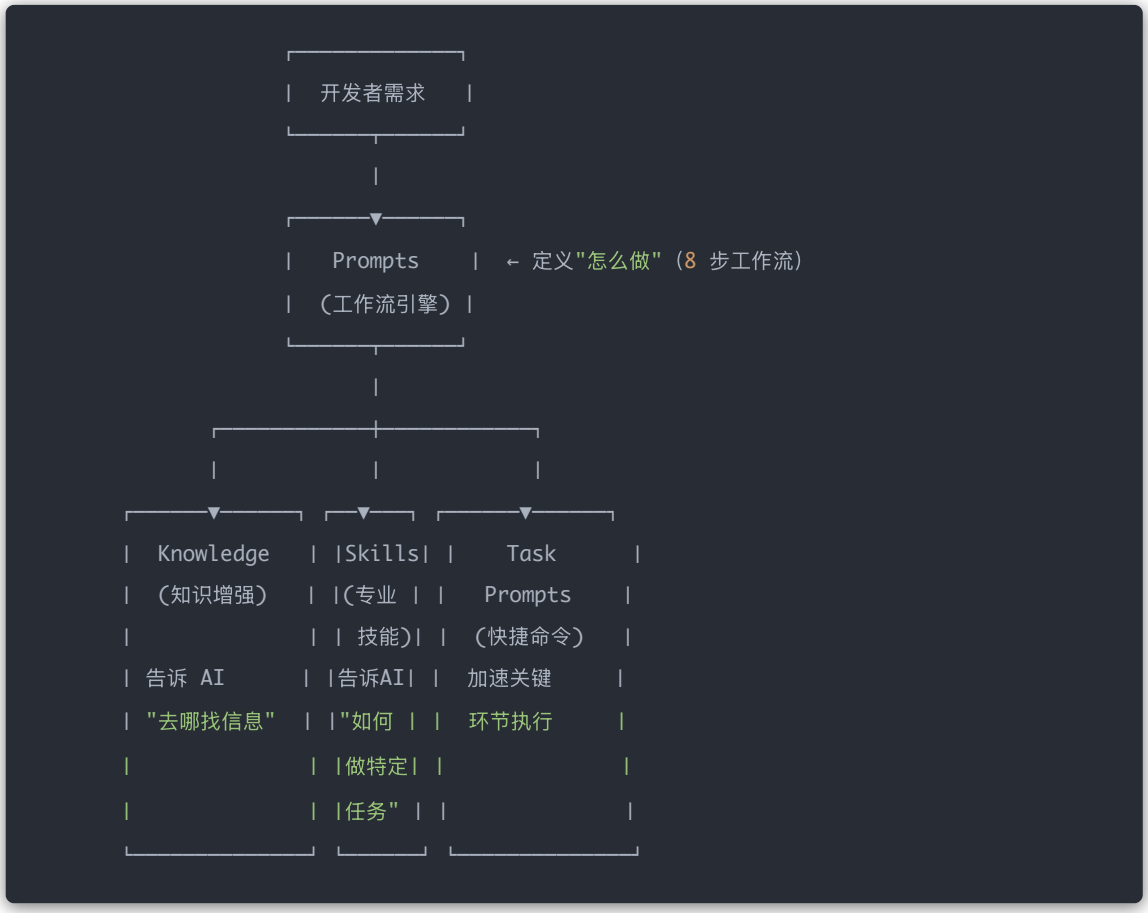
```
1. 确定指标名称（如 SplitView.Activated）
2. 定位元数据目录（tools/metrics/histograms/metadata/split_view/）
3. 更新 histograms.xml（添加 <histogram> 条目）
4. 更新 enums.xml（定义枚举值）
5. 设置过期时间（3 个月后）
6. 添加至少两个 owner
```

Knowledge 层面：`knowledge_base.md` 检测到 "UMA" 关键词，路由到 `docs/metrics/uma/README.md`，AI 读取后获得完整的 UMA 规范。

8.5 提交阶段 — Task Prompts 加速

```
/cr:git/pre-upload-checklist    → 一键执行预提交检查
/cr:gerrit/cl-description       → 自动生成 CL 描述
/cr:gerrit/fix-review-comments  → 自动修复 Review 意见
```

8.6 三大机制的协同关系



总结

Chromium 的 AI Coding 体系建立在**一条底线**和**三大机制**的精密协同之上：

1. **AI Policy（使用政策）** — 明确人与 AI 的责任边界：人类开发者对每一行代码负全责，AI 是工具而非作者

2. **Prompts（提示词）** — 四层分层组合架构，从核心指令到平台模板到任务命令，确保 AI 在任何场景下都有正确的行为规范
3. **Skills（技能）** — 18+ 个按需激活的专业模块，将复杂任务（如移除 Feature Flag 的 17 个步骤）编码为可复用的 checklist，确保不遗漏
4. **Knowledge Base（知识库）** — 三层 Agentic RAG 架构，从静态路由表到动态文档搜索到外部知识源，确保 AI 基于权威文档而非通用知识工作

此外，完善的 **Eval 评估体系**（15+ 个评估用例 + 自动化测试框架）确保了整套 AI Coding 基础设施在持续迭代中保持稳定可靠，不会因提示词修改而导致 AI 行为退化。

Chromium 的文档如此详尽，也并非一天两天的事情。查看提交记录可以发现，文档从 2015 年开始积累，跨越 11 年，总计 6445 次提交，逐步沉淀至今。`agents/` 目录于 2025 年 7 月 10 日创建，而 chromium-docs 这个核心 Skill 则是 2026 年 1 月由一位微软工程师提交的。

对于我们团队而言，如何积累出同等深度的文档体系，是落地 AI Coding 的关键挑战。

需要说明的是，本文仅从工程代码层面进行分析，并非 Chromium 真实完整的开发流程。



腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
688篇原创内容
公众号



阅读 1.2万

留言

写留言

还没有留言，来写首评